

دستگاه توپ

مدینه برای سرگرم کردن شرکت‌کنندگان در مسابقات جهانی شطرنج، یک دستگاه توپ اختراع کرده است. ساختار داخلی دستگاه به شرح زیر است:

- این دستگاه از N راس تشکیل شده است که از 0 تا $N - 1$ شماره‌گذاری شده‌اند. راس $N - 1$ ریشه نامیده می‌شود.

- برای هر راس u با $0 \leq u < N - 1$ ، پدر راس u راس $P[u]$ با $P[u] > u$ است و راس u فرزند راس $P[u]$ است. ریشه پدری ندارد. راسی که فرزندی ندارد، برگ نامیده می‌شود.

شما مقدار N و پدر هر راس u یعنی $P[u]$ را نمی‌دانید. در عوض، مدینه مقدار M یعنی تعداد برگ‌ها، و این‌که شماره‌ی برگ‌ها $0, 1, \dots, M - 1$ است را به شما می‌گوید. وظیفه‌ی شما تعیین ساختار داخلی دستگاه از طریق کار با آن است.

هر راس از دستگاه می‌تواند حداکثر یک توپ را در خود نگه دارد و هر توپ دارای مقداری است که یک عدد صحیح غیر منفی است. می‌گوییم یک راس دارای مقدار x است، اگر تویی با مقدار x را در خود نگه دارد. یک راس اگر حاوی توپ باشد، اشغال شده است؛ در غیر این صورت، آزاد است. در ابتدا، تمام راس‌ها آزاد هستند.

شما می‌توانید دو نوع عملیات انجام دهید:

- insert(U, x)**: تلاش برای درج یک توپ جدید با مقدار x در برگ U .

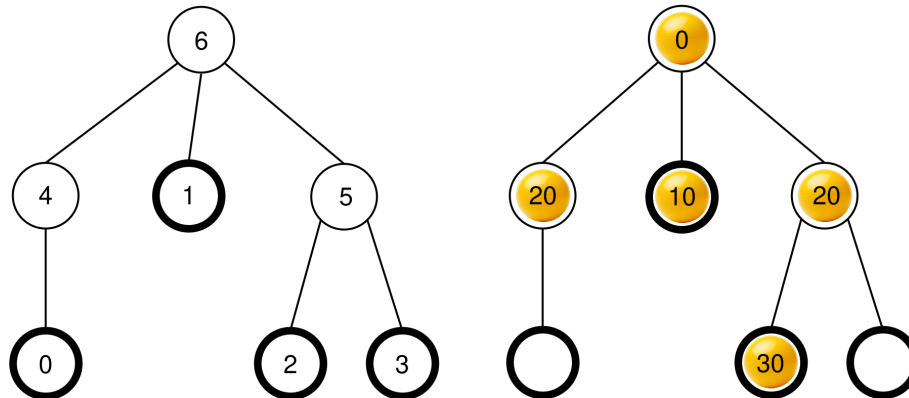
- اگر برگ U اشغال شده باشد، عملیات بدون قرار دادن توپ، مقدار $false$ را برمی‌گرداند.
- اگر برگ U آزاد باشد، توپ در راس U قرار می‌گیرد. سپس توپ به طور مکرر به سمت پدر راس فعلی خود حرکت می‌کند، تا زمانی که آن پدر آزاد باشد. حرکت زمانی متوقف می‌شود که توپ به ریشه یا راسی که پدر آن اشغال شده است، برسد. این عملیات مقدار $true$ را برمی‌گرداند.

- collect()**: اگر ریشه آزاد باشد (که به معنی خالی بودن دستگاه است)، **collect** یک آرایه‌ی خالی برمی‌گرداند. در غیر این صورت، تابع بازگشتی **traverse** که در زیر توضیح داده شده است، مقادیر تمام توپ‌هایی را که در حال حاضر در دستگاه هستند، در یک آرایه‌ی S جمع‌آوری می‌کند. در ابتدا، آرایه‌ی S خالی است و **traverse($N - 1$)** فراخوانی می‌شود.

```
traverse(u):
    append the value of the ball at node u to the end of S
    let c[u] be the list of the occupied children of u
    sort c[u] in non-decreasing order according to the values
        of the balls at those nodes (if multiple nodes have the
        same value, they can appear in any order)
    for each v in c[u]:
        traverse(v)
```

پس از اتمام این فرآیند، تمام توپ‌ها از دستگاه خارج می‌شوند و collect مقدار S را برمی‌گرداند.

برای مثال، دستگاهی با $N = 7$ راس و آرایه‌ی $P = [4, 6, 5, 5, 6, 6]$ را در نظر بگیرید. تصویر سمت چپ، دستگاه خالی را با اندیس‌های راس نشان می‌دهد، در حالی که تصویر سمت راست، پیکربندی احتمالی توپ‌ها را پس از انجام توالی خاصی از عملیات insert نشان می‌دهد (این توالی در بخش مثال آورده شده است).



وقتی `collect()` فراخوانی می‌شود، ریشه (راس 6) اشغال شده است، بنابراین S به صورت $S = []$ مقداردهی اولیه می‌شود و `traverse(6)` فراخوانی می‌شود.

traverse(6) : راس 6 مقدار 0 دارد؛ اضافه کردن آن به S نتیجه $S = [0]$ را می‌دهد. فرزندان راس 6 عبارتند از 4، 1 و 5 که همگی اشغال شده‌اند. مقادیر آنها به ترتیب 20، 10 و 20 است. مرتب‌سازی به ترتیب غیر نزولی نتیجه $c[6] = [1, 5, 4]$ را می‌دهد (توجه داشته باشید که $c[6] = [1, 4, 5]$ نیز معتبر خواهد بود، زیرا راس‌های 4 و 5 مقدار یکسانی دارند). سپس این تابع `traverse` را روی هر راس در $c[6]$ به آن ترتیب فراخوانی می‌کند.

- **traverse(1)** : راس 1 مقدار 10 دارد؛ اضافه کردن آن به S نتیجه می‌دهد $S = [0, 10]$. راس 1 هیچ فرزند اشغال شده‌ای ندارد، بنابراین این فراخوانی پایان می‌یابد.

- **traverse(5)** : راس 5 مقدار 20 دارد؛ اضافه کردن آن به S نتیجه می‌دهد $S = [0, 10, 20]$. راس 5 یک فرزند اشغال شده، راس 2، دارد، بنابراین $c[5] = [2]$ و تابع `traverse(2)` را فراخوانی می‌کند.

- **traverse(2)** : راس 2 مقدار 30 را دارد؛ اضافه کردن آن به S نتیجه می‌دهد $S = [0, 10, 20, 30]$. راس 2 هیچ فرزند اشغال شده‌ای ندارد، بنابراین این فراخوانی پایان می‌یابد.

- اجرا به `traverse(5)` برمی‌گردد که تمام فرزندان را در $c[5]$ پردازش کرده است، بنابراین فراخوانی آن پایان می‌یابد.

- **traverse(4)** : راس 4 مقدار 20 را دارد؛ اضافه کردن آن به S نتیجه می‌دهد $S = [0, 10, 20, 30, 20]$. راس 4 هیچ فرزند اشغال شده‌ای ندارد، بنابراین این فراخوانی پایان می‌یابد.

همه فراخوانی‌ها تکمیل شده‌اند و راس دیگری برای پردازش وجود ندارد. در نهایت، تمام توپ‌ها از دستگاه حذف می‌شوند و تابع `collect` آرایه $S = [0, 10, 20, 30, 20]$ را برمی‌گرداند.

وظیفه‌ی شما تعیین تعداد راس‌های N و گزارش آرایه‌ای از پدرهای $R = [R[0], R[1], \dots, R[N-2]]$ است که ساختار دستگاه را توصیف می‌کند، اما با برچسب‌گذاری بالقوه متفاوتی از راس‌هایی که برگ و ریشه نیستند. به بیان دقیق‌تر، یک آرایه‌ی گزارش‌شده‌ی R صحیح در نظر گرفته می‌شود، اگر بتوان یک برچسب منحصر به فرد $L[u]$ با $0 \leq L[u] < N$ به هر راس u از دستگاه اختصاص داد به طوری که:

- $L[u] = u$ برای همه $0 \leq u < M$ و $u = N - 1$
- $L[P[u]] = R[L[u]]$ برای همه $0 \leq u < N - 1$
- لازم نیست که $R[u] > u$ باشد. هنگام گزارش راه‌حل، دستگاه باید خالی باشد.

فرض کنید K تعداد عملیات collect انجام شده و B حداکثر مقداری باشد که روی هر توپ در تمام فراخوانی‌های insert قرار دارد. فرض کنید $C = K + B$. C نباید از 1000 بیشتر شود. امتیاز شما در برخی از زیر مسایل به مقدار C بستگی دارد.

جزئیات پیاده‌سازی

شما باید تابع زیر را پیاده‌سازی کنید.

```
std::vector<int> find_structure(int M)
```

- M : تعداد برگ‌های موجود در دستگاه.
- این تابع دقیقاً یک بار برای هر نمونه فراخوانی می‌شود.
- این تابع باید یک آرایه‌ی $R = [R[0], R[1], \dots, R[N - 2]]$ به طول $N - 1$ را برگرداند که یک آرایه‌ی معتبر از پدران است.

برای تعامل با دستگاه، تابع شما می‌تواند دو تابع زیر را فراخوانی کند.

تابع اول عبارت است از:

```
bool insert(int U, int X)
```

- U : اندیس برگ‌گی که توپ باید در آن قرار گیرد. شرط $0 \leq U < M$ باید برقرار باشد.
- X : مقدار صحیح توپ. شرط $0 \leq X \leq 1000$ باید برقرار باشد.
- این تابع اگر توپ با موفقیت وارد شده باشد، true و اگر برگ U قبلاً اشغال شده باشد، مقدار false را برمی‌گرداند.
- این تابع حداکثر می‌تواند 500 000 بار فراخوانی شود.

تابع دوم عبارت است از:

```
std::vector<int> collect()
```

- مقادیر تمام توپ‌های درج‌شده را جمع‌آوری می‌کند، دستگاه را خالی می‌کند و آرایه‌ی جمع‌آوری‌شده‌ی S را برمی‌گرداند.

اگر در هر مرحله از اجرای برنامه، مقدار C از 1000 بیشتر شود، راه‌حل شما با این خطا مواجه خواهد شد: Output isn't correct: Too many resources used

ساختار دستگاه قبل از فراخوانی find_structure ثابت می‌شود. ارزیاب از نوع معین است؛ به این معنی که اگر آن را دو بار اجرا کنید و هر دو اجرا همان عملیات را انجام دهند، فراخوانی‌های collect همان آرایه‌ها را باز می‌گردانند.

محدودیت‌ها

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

زیرمسئله‌ها

Subtask	Score	Additional Constraints
1	5	The root has exactly M children.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	No additional constraints.

در زیرمسائل ۳ و ۴، امتیاز شما به مقدار C به شرح زیر بستگی دارد.

زیرمسئله‌ی ۳ (۲۵ امتیاز)

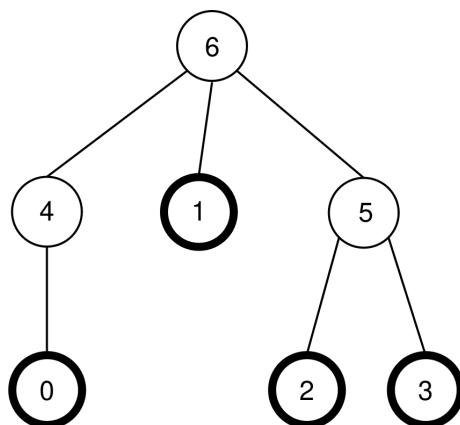
Limits	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

زیرمسئله‌ی ۴ (۶۰ امتیاز)

Limits	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

مثال

همان دستگاهی را که در شرح مسئله آمده است در نظر بگیرید، بنابراین $N = 7$ ، $M = 4$ و $P = [4, 6, 5, 5, 6, 6]$. دستگاه مجدداً در شکل زیر نشان داده شده است:



ارزیاب فراخوانی زیر را انجام می‌دهد:

```
find_structure(4)
```

این تابع می‌تواند دنباله‌ی فراخوانی‌های زیر را انجام دهد:

۱. `insert(0, 0)` : برگ 0 آزاد است، بنابراین یک توپ با مقدار 0 در برگ 0 قرار می‌گیرد. راس $P[0] = 4$ آزاد است، بنابراین توپ به راس 4 حرکت می‌کند. راس $P[4] = 6$ آزاد است، بنابراین توپ به راس 6 حرکت می‌کند. از آنجایی که راس 6 ریشه است، حرکت متوقف می‌شود. فراخوانی مقدار `true` را برمی‌گرداند.

۲. `insert(3, 20)` : برگ 3 آزاد است، بنابراین توپی با مقدار 20 در برگ 3 قرار می‌گیرد. راس $P[3] = 5$ آزاد است، بنابراین توپ به راس 5 حرکت می‌کند. از آنجایی که $P[5] = 6$ اشغال شده است، حرکت متوقف می‌شود. فراخوانی مقدار `true` را برمی‌گرداند.

۳. `insert(1, 10)` : برگ 1 آزاد است، بنابراین یک توپ با مقدار 10 در برگ 1 قرار می‌گیرد. از آنجایی که $P[1] = 6$ اشغال شده است، توپ در راس 1 باقی می‌ماند. فراخوانی مقدار `true` را برمی‌گرداند.

۴. `insert(2, 30)` : برگ 2 آزاد است، بنابراین یک توپ با مقدار 30 در برگ 2 قرار می‌گیرد. از آنجایی که $P[2] = 5$ اشغال شده است، توپ در راس 2 باقی می‌ماند. فراخوانی مقدار `true` را برمی‌گرداند.

۵. `insert(1, 25)` : برگ 1 اشغال شده است، بنابراین توپ در برگ 1 قرار نمی‌گیرد. فراخوانی مقدار `false` را برمی‌گرداند.

۶. `insert(0, 20)` : برگ 0 آزاد است، بنابراین توپی با مقدار 20 در برگ 0 قرار می‌گیرد. راس $P[0] = 4$ آزاد است، بنابراین توپ به راس 4 حرکت می‌کند. از آنجایی که $P[4] = 6$ اشغال شده است، حرکت متوقف می‌شود. فراخوانی مقدار `true` را برمی‌گرداند.

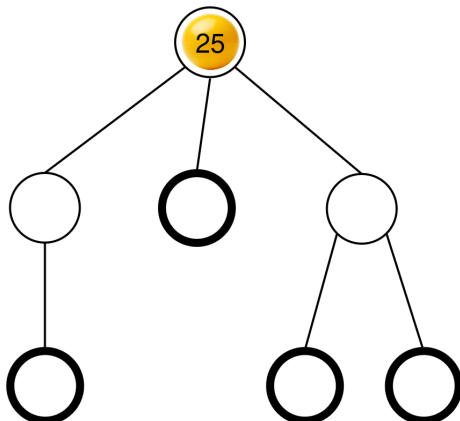
پیکربندی حاصل از توپ‌ها قبلاً در شرح مسئله نمایش داده شده بود.

در مرحله‌ی بعد، این تابع می‌تواند موارد زیر را فراخوانی کند:

```
collect()
```

اجرای این فراخوانی قبلاً در توضیحات مسئله توضیح داده شده است، در نتیجه این فراخوانی $S = [0, 10, 20, 30, 20]$ را برمی‌گرداند. پس از آن، دستگاه دوباره خالی می‌شود.

در مرحله‌ی بعد، این تابع می‌تواند تابع $\text{insert}(2, 25)$ را فراخوانی کند. برگ 2 آزاد است، بنابراین یک توپ با مقدار 25 در برگ 2 قرار می‌گیرد. این توپ سپس به راس 5 و سپس به راس 6 حرکت می‌کند و در آن‌جا متوقف می‌شود. این فراخوانی مقدار `true` را برمی‌گرداند. پیکربندی حاصل از توپ‌ها در شکل زیر نشان داده شده است.



در نهایت، این تابع می‌تواند تابع $\text{collect}()$ را فراخوانی کند که $S = [25]$ را برمی‌گرداند و دستگاه را خالی می‌کند.

دو آرایه‌ی پدر معتبر وجود دارند که find_structure می‌تواند برگرداند: $R = P = [4, 6, 5, 5, 6, 6]$ (با شماره‌گذاری $L = [0, 1, 2, 3, 4, 5, 6]$) و $R = [5, 6, 4, 4, 6, 6]$ (با شماره‌گذاری $L = [0, 1, 2, 3, 5, 4, 6]$). در این مثال، $K = 2$ و $B = 30$ است. در نتیجه $C = 32$ می‌شود.

ارزیاب نمونه

قالب ورودی:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

قالب خروجی:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

در این‌جا، Q طول آرایه‌ی R است که توسط find_structure برگردانده شده است.